

WEEK 10 HOMEWORK SUBMISSION

Campus Service Request System

OOP Best Practices, SRP, and MVC Preparation

Student Name	Nguyễn Thu Hồng
Class	INS306402-INS3064
Course	Multimedia Design and Web Development
Submission Type	PDF/Word report + optional PHP skeleton ZIP

Source: Week 10 Worksheet - One-Week Homework Worksheet (MIS-friendly).

Contents

1. 1. Scenario and Problem Understanding
2. 2. Day 1 - Requirements List and User Roles
3. 3. Day 2 - Use Cases
4. 4. Day 3 - SRP Class Responsibility Table
5. 5. Day 4 - UML Class Diagram
6. 6. Day 5 - MVC Mapping Table and Route Table
7. 7. Day 6 - Minimal Skeleton and Folder Structure
8. 8. Day 7 - Reflection Answers
9. 9. Final Submission Checklist

1. Scenario and Problem Understanding

Scenario. This homework designs a simple Campus Service Request system. Students can submit campus-related requests, such as a broken projector, a broken chair, or a request for access to the MIS lab. Staff members can review submitted requests and update their processing status.

Core features. The system keeps the scope small and focuses on four features: submitting a request, viewing a request list, viewing request details, and updating request status from Pending to In Progress and Done.

Design focus. The main purpose is not to build a full production system. The purpose is to practice OOP design, the Single Responsibility Principle (SRP), basic type thinking, naming conventions, and MVC preparation for next week.

2. Day 1 - Requirements List and User Roles

2.1 Requirements List

- The system shall allow a student to submit a new campus service request.
- The request form shall collect a title, description, room/location, and creator information.
- The title field shall be required and cannot be empty.
- The description field shall be required and cannot be empty.
- The room/location field shall be required and cannot be empty.
- Each new request shall have the default status Pending.
- The system shall display a list of service requests.
- The request list shall show the request ID, title, room/location, and status.
- The system shall allow users to view detailed information for one request.
- The request details page shall show title, description, room/location, status, and creator role.
- The system shall allow staff to update the status of a request.
- The allowed statuses shall be Pending, In Progress, and Done.
- The system shall reject invalid request status values.
- The system shall separate data logic, business rules, controller logic, and view files.
- The system shall follow clear naming conventions such as PascalCase for classes and camelCase for methods.

2.2 User Roles

Role	Description	Main Permissions
Student	A campus user who submits requests when there is a problem or need.	Create requests; view request list; view request details.
Staff	A staff member who reviews and handles service requests.	View all requests; view request details; update request status.

3. Day 2 - Use Cases

3.1 Use Case: Submit a Request

Item	Description
Use Case Name	Submit a Request
Actor	Student
Trigger	The student finds a campus facility problem or needs support.
Main Flow	1. Student opens the request creation form. 2. Student enters the request title, description, and room/location.

	3. Student clicks the Submit button. 4. The system validates the input data. 5. The system saves the request with Pending status. 6. The system redirects the student to the request list.
Alternative Flow	If required fields are empty, the system displays validation errors and does not save the request.
Success Outcome	A new service request is created successfully with Pending status.

3.2 Use Case: View Request List

Item	Description
Use Case Name	View Request List
Actor	Student / Staff
Trigger	The user wants to see submitted service requests.
Main Flow	1. User opens the request list page. 2. The controller receives the request. 3. The controller calls the service to get all requests. 4. The service retrieves request data from the repository. 5. The system displays the list of requests in a table.
Alternative Flow	If there are no requests, the system can display an empty list message.
Success Outcome	The user can see all available requests with basic information.

3.3 Use Case: View Request Details

Item	Description
Use Case Name	View Request Details
Actor	Student / Staff
Trigger	The user selects one request from the list.
Main Flow	1. User clicks View Details for a request. 2. The controller receives the request ID. 3. The controller calls the service to find the request. 4. The service asks the repository for the selected request. 5. The system displays the detail page for that request.
Alternative Flow	If the request ID does not exist, the system shows a Request not found message.
Success Outcome	The user can view the full title, description, room/location, status, and creator information.

3.4 Use Case: Update Request Status

Item	Description
Use Case Name	Update Request Status
Actor	Staff
Trigger	A staff member starts handling or finishes a service request.
Main Flow	1. Staff opens the request detail page. 2. Staff selects a new status from the allowed status list. 3. Staff submits the update form. 4. The controller sends the new status to the service. 5. The service validates whether the status is allowed. 6. The repository updates the request status.
Alternative Flow	If the status is invalid, the system rejects the update.
Success Outcome	The request status is updated to Pending, In Progress, or Done.

4. Day 3 - SRP Class Responsibility Table

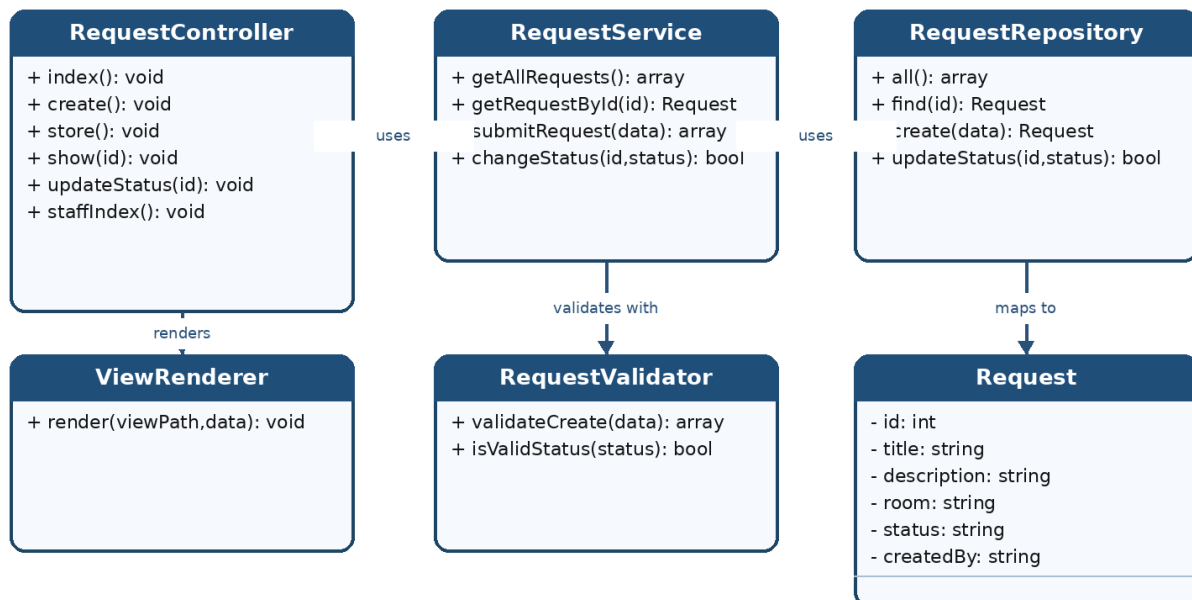
The table below follows the Single Responsibility Principle. Each class has one main responsibility and one main reason to change.

Class name	Responsibility (1 sentence)	Reason to change (1 reason only)
Request	Stores request data such as ID, title, description, room/location, status, and creator role.	Request data fields change.
RequestRepository	Reads and writes request data from the storage layer.	Storage method changes, for example from session storage to MySQL.
RequestService	Applies business rules such as allowed status changes and request submission flow.	Business rules change.
RequestValidator	Validates input data for creating requests and updating status.	Validation rules change.
RequestController	Receives user actions, calls the service, and selects the correct view.	Routes or controller actions change.
ViewRenderer	Loads view files and passes variables to HTML templates.	Rendering method or view loading logic changes.

5. Day 4 - UML Class Diagram

The UML diagram shows the basic relationship between the main classes. RequestController uses RequestService. RequestService uses RequestRepository and RequestValidator. RequestRepository maps stored data into Request objects.

Simple UML Class Diagram - Campus Service Request System



Key idea: Controller coordinates requests, Service applies business rules, Repository accesses data, Request stores data, Validator checks input, ViewRenderer renders views.

Figure 1. Simple UML Class Diagram for Campus Service Request System

6. Day 5 - MVC Mapping Table and Route Table

6.1 MVC Mapping Table

Item	MVC Layer	Why?
Request	Model	Represents the application data and status of a campus service request.
RequestRepository	Model	Accesses the data source and hides storage details from the controller.
RequestService	Model / Service	Contains business rules and coordinates request operations.
RequestValidator	Model / Service	Checks whether input data is valid before saving or updating.
RequestController@index	Controller	Receives the list request and chooses the index view.
RequestController@create	Controller	Shows the form for creating a request.
RequestController@store	Controller	Receives submitted form data and calls the service to save it.
RequestController@show	Controller	Receives a request ID and selects the detail view.
RequestController@updateStatus	Controller	Receives staff status update input and calls the service.
requests/index.php	View	Displays the request list UI.
requests/create.php	View	Displays the request creation form.
requests/show.php	View	Displays the request detail UI and status update form.

6.2 Route Table

HTTP	URL	Controller@Action	Purpose
GET	/requests	RequestController@index	List all service requests.
GET	/requests/create	RequestController@create	Show the create request form.
POST	/requests	RequestController@store	Save a new request.
GET	/requests/{id}	RequestController@show	View request details.
POST	/requests/{id}/status	RequestController@updateStatus	Update request status.
GET	/staff/requests	RequestController@staffIndex	Show staff request dashboard.

7. Day 6 - Minimal Skeleton and Folder Structure

The optional PHP skeleton follows the MVC idea and can be placed inside C:\xampp\htdocs to run with XAMPP. It uses session-based storage, so MySQL is not required for this week.

7.1 Folder Structure

```
campus_service_request_week10_full/
|-- app/
|   |-- Controllers/
|   |   `-- RequestController.php
|   |-- Core/
|   |   `-- ViewRenderer.php
|   |-- Models/
|   |   |-- Request.php
|   |   `-- RequestRepository.php
|   |-- Services/
|   |   `-- RequestService.php
|   |-- Validators/
```

```
| | `-- RequestValidator.php
| `-- Views/
|   |-- requests/
|     |-- index.php
|     |-- create.php
|     |-- show.php
|-- public/
| `-- index.php
`-- README.md
```

7.2 Pseudocode

RequestController@store():

- read title, description, and room from POST
- call RequestService.submitRequest(data)
- if validation fails, show create view with errors
- if validation succeeds, redirect to /requests

RequestController@updateStatus(id):

- read newStatus from POST
- call RequestService.changeStatus(id, newStatus)
- redirect to /requests/{id}

RequestService.changeStatus(id, newStatus):

- check whether newStatus is Pending, In Progress, or Done
- if status is valid, call RequestRepository.updateStatus(id, newStatus)
- return true or false

8. Day 7 - Reflection Answers

1. Which parts of your design are Model, View, and Controller?

The Model part includes Request, RequestRepository, RequestService, and RequestValidator because these classes handle application data, data access, validation, and business rules. The View part includes the files inside app/Views/requests, such as index.php, create.php, and show.php, because these files display HTML output to users. The Controller part is RequestController because it receives user actions, calls the service layer, and chooses which view should be shown.

2. Where should validation happen, and why?

Validation should happen in RequestValidator, not directly inside the View file. This keeps the View focused only on displaying the interface. It also makes validation rules easier to maintain. For example, if the system later requires a longer description or a specific room format, only RequestValidator needs to change.

3. What would break if you put SQL inside a View file?

If SQL is placed inside a View file, the system mixes presentation logic with data access logic. This breaks the MVC structure and makes the code difficult to maintain. A small database change could force developers to edit HTML files. It also makes the application harder to test, less secure, and less reusable.

4. What code do you expect to write next week to make this real?

Next week, the design can be converted into a working MVC mini application by implementing a real Router, writing controller actions based on the route table, connecting RequestRepository to a MySQL database using PDO, creating real views for list/detail/create pages, and using autoloading so classes can be loaded automatically.

9. Final Submission Checklist

- Requirements list is included.
- User roles are included.
- At least four use cases are included.
- SRP Class Responsibility Table has at least six classes.
- UML class diagram is included as a figure.
- MVC Mapping Table is included.
- Route Table has at least six routes.
- Reflection answers are included.
- Optional PHP skeleton ZIP is prepared for XAMPP.

Conclusion. This submission satisfies the Week 10 worksheet by combining analysis, use cases, SRP-based class design, UML diagram, MVC mapping, route planning, reflection, and an optional PHP MVC skeleton for XAMPP.